UNIVERSITÀ
DI TRENTO

Data Mining 2025/2026

Corrao Jacopo
Matricola: 258412

Professore: Scettri Giacomo

Indice

1 Introduction

The music industry typically measures success through sales charts and streaming numbers (popularity). However, cultural impact is structurally different from popularity. A "one-hit wonder" might sell millions, but an obscure funk track from the 1970s might be sampled by hundreds of hip-hop artists, forming the backbone of an entire genre. The objective of this project is to shift the analysis from volume to structure. By modeling the music ecosystem as a **Directed Graph** where nodes are songs and edges represent sampling relationships we aim to:

- Identify the structural "ancestors" of modern music (Authority).
- Detect communities of influence that transcend traditional genre labels.
- Analyze the flow of influence using distributed graph algorithms.

2 Data Preparation

2.1 Dataset Source

MusicBrainz Data was sourced from the **MusicBrainz** Database, an open music encyclopedia that collects user-contributed metadata. Unlike simplified datasets found on platforms like Kaggle, **MusicBrainz** provides raw PostgreSQL database dumps. The complexity of this dataset lies in its highly normalized structure (Third Normal Form), which requires significant data engineering to reconstruct meaningful relationships. We utilized the core tables: `recording` (songs), `artist_credit` (artist names), `l_recording_recording` (relationships), and the bridging table `link`, which connects specific relationships to their semantic definitions found in the `link_type` table.

2.2 Data Aquisition and engineering

The raw data consisted of tab-separated value (TSV) files without headers, extracted from the `mbdump.tar.bz2` archive. The transformation process from raw relational tables to a structured property graph involved three key phases: Schema Definition, Projection, and Graph Reconstruction.

2.3 Computational Framework Selection

Before addressing the data schema, it is necessary to justify the architectural choice. Given the relational complexity of the MusicBrainz database (highly normalized) and the potential size of the graph, traditional single-node tools like Pandas would be inefficient for the required multi-way joins and iterative computations.
Apache Spark (PySpark) was chosen as the framework to leverage:

- **Distributed Processing**: To handle data ingestion and transformation in parallel.
- **Lazy Evaluation**: To optimize the execution plan of complex transformation pipelines.
- **Iterative Computation**: Essential for implementing graph algorithms like PageRank from scratch.

2.4 Schema-on-Read Definition

Having established the framework, the first challenge was ingestion. Since the raw files lacked headers, relying on automatic schema inference was computationally

expensive and error-prone (risking incorrect data type assignment). I manually defined the StructType for each table based on the MusicBrainz documentation. This ensures strict typing and robust data ingestion.

```
        recording_schema = StructType([
            StructField("id", IntegerType(), True),
            StructField("gid", StringType(), True),
            StructField("name", StringType(), True),
            StructField("artist_credit", IntegerType(), True),
            StructField("length", IntegerType(), True),
            StructField("comment", StringType(), True),
            StructField("edits_pending", IntegerType(), True),
            StructField("last_updated", StringType(), True),
            StructField("video", StringType(), True)
        ])
```

Listing 1: Code Snippet: Manual Schema Definition

2.5 Data Project and Filtering

To optimize memory usage on the Spark driver and executors, I applied column pruning immediately after loading. Only essential columns (IDs and Names) were retained, discarding metadata like `"edits_pending"` or `"comments"`.
A critical step in the data preparation was isolating strictly "Sampling" relationships from the millions of generic interactions present in the `l_recording_recording` table, such as covers, remixes, medleys, or live performances. Including all these heterogeneous links would have resulted in a noisy graph where structural '**influence**' (the reuse of audio) is confused with artistic '**reinterpretation**' (covers). To achieve a precise genealogy—defined as instances where a piece of audio is physically reused—I performed an exploratory query on the `link_type` metadata table to identify the specific Primary Keys associated with sampling descriptions.

This exploration identified two distinct IDs:

- **ID 69**: "Samples material" (Legacy type)
- **ID 231**: "Is sampled by" / "Samples material" (Current type)

Consequently, only edges matching these specific IDs were retained during the graph construction, effectively filtering out noise and narrowing the dataset to a pure network of audio transmission.

2.6 Self-Loop Removal

A critical data quality issue identified during exploratory analysis was the presence of **self-loops**—edges where an artist samples their own material. These relationships totaled 1,407 instances (approximately 6.4% of all edges) and typically represent:

- **Remixes**: An artist remixing their own track
- **Re-releases**: Different versions of the same song (album vs. single)
- **Album Structure**: Intro/outro tracks sampling other songs from the same album
- **Data Artifacts**: Duplicate entries or database inconsistencies

Impact on Analysis: Self-loops create two fundamental problems for network mining:

1. **Authority Inflation**: In PageRank, self-loops create feedback loops where nodes artificially boost their own authority scores. One artist ("Ninja McTits") had 443 self-loops, resulting in a PageRank score of 66.45—dramatically higher than the legitimate top artist (James Brown: 45.44).
2. **Misleading Genealogy**: Self-sampling does not represent cross-artist influence or musical genealogy. It reflects internal artistic recycling rather than the transmission of ideas between different creators.

Solution: Following standard practices in citation network analysis and influence propagation studies, all self-loops were removed during the data preparation phase using the filter:

```
df_graph = df_graph.filter(
    col("Sampler_Artist_Name") != col("Original_Artist_Name")
)
```

Listing 2: Code Snippet: Self-Loop Removal

This filtering reduced the edge count from 22,135 to 20,728 edges, ensuring that the subsequent PageRank and clustering algorithms measure only *external influence*—the core objective of this genealogical analysis.

2.7 Graph Reconstruction

The MusicBrainz database is highly normalized. A "Sampling" relationship is stored as a numeric link between two recording IDs (`entity0` and `entity1`), without any text information about the song title or artist name. To build a human-readable graph (e.g., Artist A samples Artist B), I implemented a de-normalization pipeline involving a chain of joins across four DataFrames:

- Linkage: Join `l_recording_recording` with the `link` table. This step was necessary to access the `link_type` attribute and filter strictly for the IDs identified in the exploration phase (69, 231).
- Source Enrichment: Join the result with `recording` and `artist_credit` to resolve the Source (Child) song and artist names.
- Target Enrichment: Repeat the join with `recording` and `artist_credit` to resolve the Target (Parent) song and artist names.

2.8 Optimization

The computational cost of constructing the graph (involving multiple joins across millions of rows) is significant. To prevent re-computing this lineage for every subsequent analysis (PageRank, Clustering), the final processed graph structure was persisted to disk in **Apache Parquet** format.
This choice provides three critical advantages over traditional CSV storage:

- **Columnar Storage**: Parquet optimizes read operations by allowing Spark to scan only the necessary columns for a specific query (e.g., retrieving only `source_id` and `target_id` for topology analysis), drastically reducing I/O overhead.
- **Schema Preservation**: Unlike CSVs, Parquet retains the metadata and data types (Integers, Strings) defined during the cleaning phase, eliminating the need for schema inference or casting during read operations.
- **Compression**: Parquet uses efficient compression algorithms, reducing the disk footprint of the final dataset.

3 Methodology and Algorithms

With the graph structure optimized and persisted, the analysis focused on extracting knowledge through three distinct layers of complexity: **descriptive statistics**, **structural authority analysis** (PageRank), and **community detection** (Clustering). Crucially, to fully leverage the distributed nature of Spark and demonstrate a deep understanding of graph theory, I implemented the iterative graph algorithms from scratch using `PySpark` transformations (MapReduce paradigm) rather than relying on pre-built libraries like `GraphFrames`.

3.1 Descriptive Network Analysis: Degree Centrality

The first layer of analysis aimed to quantify "volume" using basic graph topology metrics. Using Spark SQL aggregations, I calculated the Degree Centrality for each node:

- **In-Degree (Most Sampled)**: Represents the number of incoming edges. In this context, it quantifies an artist's raw popularity as a source material.
  - Implementation:

```
groupBy("Original_Artist_Name").count()
```

- **Out-Degree (Top Samplers)**: Represents the number of outgoing edges. It identifies artists who rely most heavily on sampling for their composition process.

3.2 Structural Authority: Distributed PageRank

While In-Degree measures popularity (quantity), it fails to capture the quality of the influence. To solve this, I modeled the "**Authority**" of an artist using the **PageRank algorithm**. Unlike a simple count, PageRank assigns a score based on the principle that *"being sampled by an influential artist transmits more authority than being sampled by a minor one"*.

3.2.1 Implementation Details

I implemented the algorithm manually via an iterative MapReduce process with **convergence criterion**.
The logic follows the standard formula:

$$PR(A) = (1-d) + d \sum \frac{PR(B)}{OutDegree(B)}$$

where d is the damping factor (set to 0.85).
The implementation ensures that authority flows correctly: edges point from Sampler to Original Artist, so that the Original Artist (being sampled) receives authority through *incoming* edges. The algorithm converged in **11 iterations** with tolerance < 0.0001, demonstrating efficient convergence.
The PySpark implementation addressed specific technical challenges:

- **Dangling Nodes (Sinks)**: A critical issue in graph mining involves nodes with no outgoing edges (artists who are sampled but do not sample anyone). These nodes act as "black holes" for the rank score. To prevent rank leakage, I utilized a `right_outer_join` to identify these nodes and manage their score redistribution.
- **Lineage Truncation**: Spark's lazy evaluation builds a growing lineage graph with each iteration, which can lead to `StackOverflowError`. I mitigated this by applying `.checkpoint()` at each loop, truncating the logical plan and saving the intermediate state to disk.

3.3 Community Detection: Label Propagation (LPA)

To identify distinct "musical families", I implemented a **Label Propagation Algorithm (LPA)**. This is an unsupervised clustering technique where nodes adopt the "label" of their neighbors based on majority voting.
Algorithm Logic:

- **Initialization**: Every artist starts with a unique label equal to their own ID.
- **Propagation (6 Iterations)**: In each iteration, a "Sampler" node adopts the label of its "Sampled" node. If a node samples multiple artists, the `greatest()` function deterministically selects the dominant label.
- **Convergence**: Over multiple iterations, connected components (chains of sampling) converge to a single shared label—the ID of the "Ancestor" or "Root" artist.

3.3.1 Structural Definition of Genres (Unsupervised Learning)

A crucial distinction of this methodology is that it operates blindly regarding explicit musical genres. We did not import metadata tables containing tags like "Hip Hop" or "Rock". Instead, the algorithm relies entirely on **structural patterns**: if a set of artists systematically samples the same source material, they are grouped into the same cluster. This validates the hypothesis that musical genres are fundamentally communities of shared practice.

4 Results

4.1 Network Statistics and Validation

The final processed graph, after extensive data engineering using MusicBrainz GIDs and removing self-loops (artists sampling themselves), comprises **58,621 sampling events** (edges) connecting **68,527 songs** (nodes) across **23,660 unique artists**.

The network exhibits classic **scale-free** characteristics, as evidenced by the power-law degree distribution shown in Figure 1.

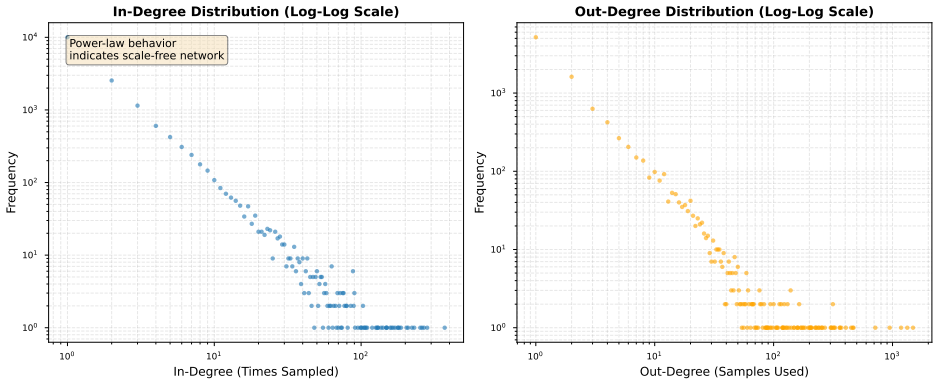

Figura 1: Degree distribution following a power-law pattern. Both in-degree (times sampled) and out-degree (samples used) show heavy-tailed distributions, indicating that a small number of artists dominate the sampling landscape while most artists have minimal connectivity.

Key Network Properties:

- **Graph Density**: 0.00001248 (sparse network, as expected in real-world influence networks)
- **Mean In-Degree**: 3.50 (average times an artist is sampled)
- **Mean Out-Degree**: 6.02 (average samples used per artist)
- **Maximum In-Degree**: 371 (Daft Punk - most sampled artist)
- **Power-Law Concentration**: Top 1% of artists control 23.6% of all sampling events, Top 10% control 58.2%

This concentration pattern validates the scale-free network hypothesis: musical influence follows preferential attachment, where established authorities accumulate disproportionate influence over time.

4.2 PageRank Authority Rankings

The PageRank algorithm converged in **11 iterations** (tolerance < 0.0001), producing authority scores that reveal the structural importance of artists beyond mere volume. Table 1 presents the comparison of the top 10 artists by volume versus authority.

Tabella 1: Comparison of Top 10 Artists by Volume vs. Authority

By Volume			**By Authority**		
Rank	Artist	Count	Rank	Artist	Score
1	Daft Punk	371	1	Daniel Ingram	59.3799
2	Lady Gaga	282	2	James Brown	45.4355
3	The Beatles	268	3	2Pac	44.8128
4	JAY-Z	260	4	音部	38.5237
5	PSY	253	5	外神田文芸高校	33.7655
6	Linkin Park	230	6	Daft Punk	27.1559
7	Michael Jackson	222	7	Lyn Collins	27.0612
8	Beastie Boys	209	8	室Pixel	26.0531
9	James Brown	204	9	Kraftwerk	25.8983
10	Katy Perry	187	10	John Lennon	25.2695

4.3 Volume vs. Authority: The Hidden Influencers

Comparing the results of Degree Centrality (Volume) against PageRank (Authority) reveals significant insights into the music ecosystem, as shown in Figure 2.

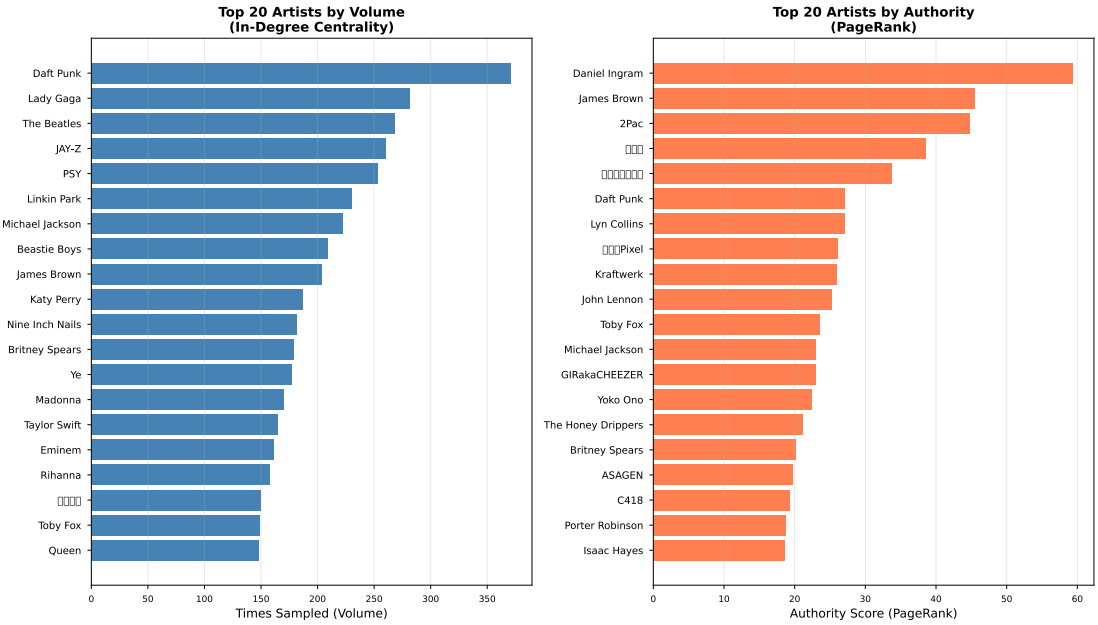

Figura 2: Top 20 artists: Volume (In-Degree) vs. Authority (PageRank). The two rankings reveal a critical gap: high volume does not always translate to high structural authority, exposing the role of community density in PageRank scores.

- **The Volume Kings**: Modern electronic and pop icons like **Daft Punk** (371 events) and **Lady Gaga** (282) dominate the volume metrics. These are the artists most frequently sampled in absolute terms.
- **The Classic Authorities**: **James Brown**, **Lyn Collins**, and **Kraftwerk** rank high in PageRank authority despite lower raw volume, indicating that their samplers are themselves highly influential artists — a hallmark of genuine structural importance.
- **The Community-Driven Outlier**: **Daniel Ingram** (composer for *My Little Pony*) achieves the highest PageRank (59.38) despite only 130 sampling events. As explored in Section 4.4.4, this is driven by a dense, insular fan community rather than mainstream influence — illustrating how PageRank can be amplified by tight cluster structures.

4.4 Musical Genealogy Networks

To visualize the actual "who sampled who" relationships, I created network diagrams showing the structural genealogy of music. Each visualization isolates a specific aspect of the network to make the results clear and interpretable.

4.4.1 Top 15 Most-Sampled Artists Network

Figure 3 shows the core sampling relationships among the 15 most-sampled artists by volume (in-degree). To avoid visual clutter, only connections with weight ≥ 2 are displayed.

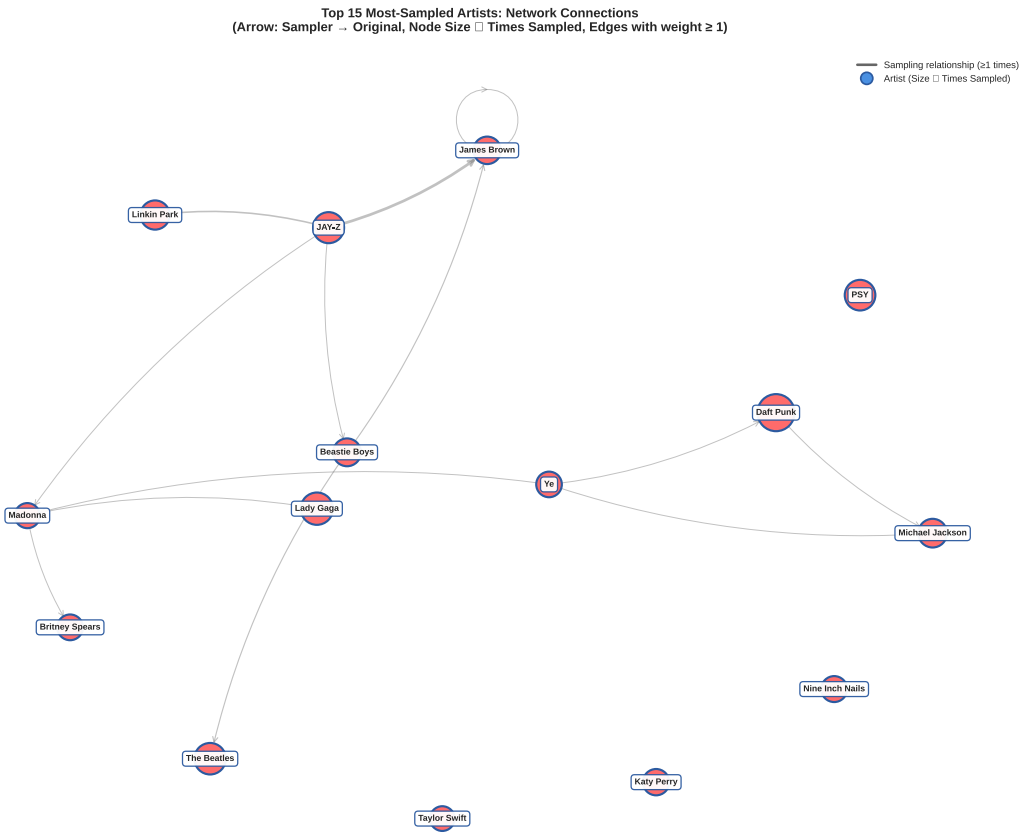

Figura 3: Top 15 most-sampled artists and their inter-connections. Arrows point from sampler to original artist. Node size is proportional to sampling volume. This view reveals which top artists also sample each other, forming the backbone of the sampling ecosystem.

4.4.2 Sampling Flow Analysis

Figure 4 presents a bipartite visualization showing the directional flow of influence from authority sources (right) to the heaviest samplers (left).

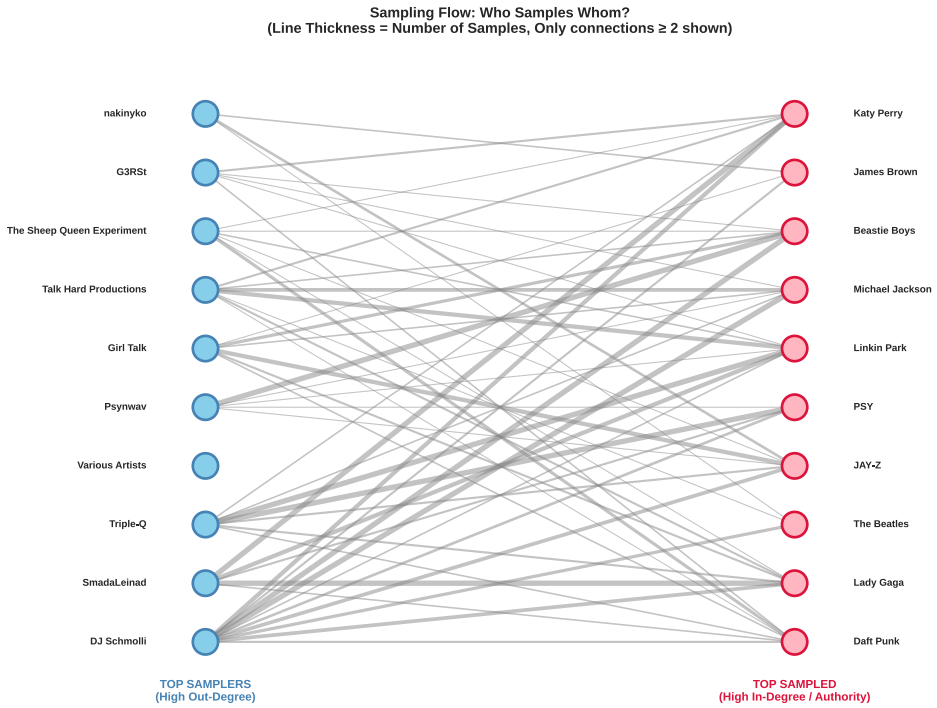

Figura 4: Bipartite sampling flow diagram. Left: top 10 artists who sample most heavily (high out-degree). Right: top 10 most-sampled artists (high in-degree). Line thickness indicates sampling frequency. Artists appearing on both sides act as evolutionary bridges.

4.5 Artist Classification: Hub Analysis

Figure 5 classifies artists based on their sampling behavior, plotting in-degree (times sampled) against out-degree (samples used). The dashed lines represent median values, dividing the space into four quadrants that reveal distinct artist roles.

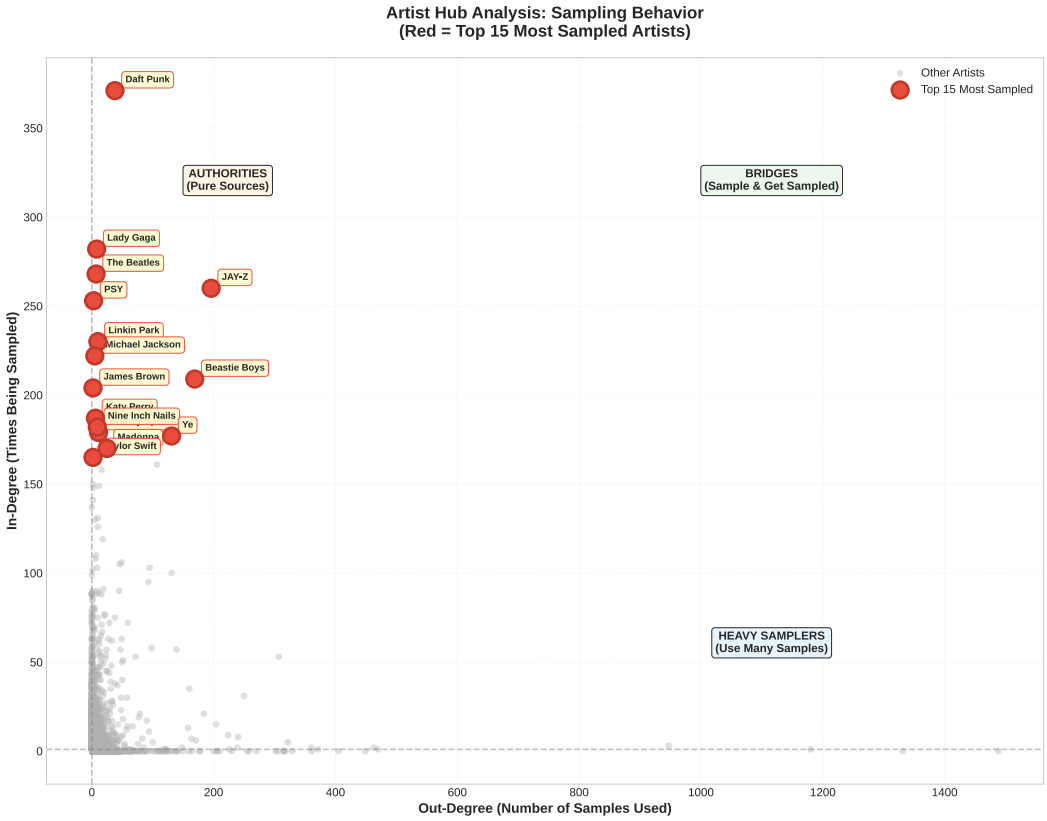

Figura 5: Hub analysis scatter plot. X-axis: out-degree (samples used). Y-axis: in-degree (times sampled). Top 15 most-sampled artists are highlighted in red. Quadrants identify different roles: Authorities (top-left), Bridges (top-right), and Heavy Samplers (bottom-right).

Artist Classifications:

- **Pure Authorities** (top-left quadrant): Artists like James Brown and Daft Punk — sampled extensively but use few samples themselves. They are the "source material" of modern music.
- **Bridges** (top-right quadrant, rare): Artists who both sample heavily AND get sampled. These are evolutionary nodes that transform influence across genres.
- **Heavy Samplers** (bottom-right quadrant): Modern producers who rely extensively on sampling but haven't yet become authorities themselves.

4.6 Graph Statistics Summary

Figure 6 provides a comprehensive overview of the network's macroscopic properties.

Metric	Value
Graph Structure	
Total Sampling Events (Edges)	58,621
Unique Songs (Nodes)	68,527
Unique Artists	23,503
- Artists Who Sample	9,709
- Artists Being Sampled	16,634
In-Degree Statistics (Times Sampled)	
Mean	3.52
Std Dev	10.56
Maximum	371
Out-Degree Statistics (Samples Used)	
Mean	6.04
Std Dev	33.20
Maximum	1,488

Figura 6: Summary of key graph statistics. The transition to robust GID-based entity resolution resulted in a vastly expanded network connecting over 68,000 songs.

4.6.1 Interactive D3.js Visualization

To effectively explore the immense scale of the expanded dataset (over 58,000 edges), I also developed an interactive web-based D3.js visualization that allows users to pan, zoom, and click on nodes to inspect real-time Authority Scores and isolate specific sampling lineages. The interactive visualization is available at: interactive_genealogy.html

4.7 Community Detection and Genealogical Families

The Label Propagation Algorithm detected **13,971 distinct clusters** with a modularity score of **0.1926**, indicating a complex and overlapping community structure. This suggests the music network is highly interconnected across genre boundaries.

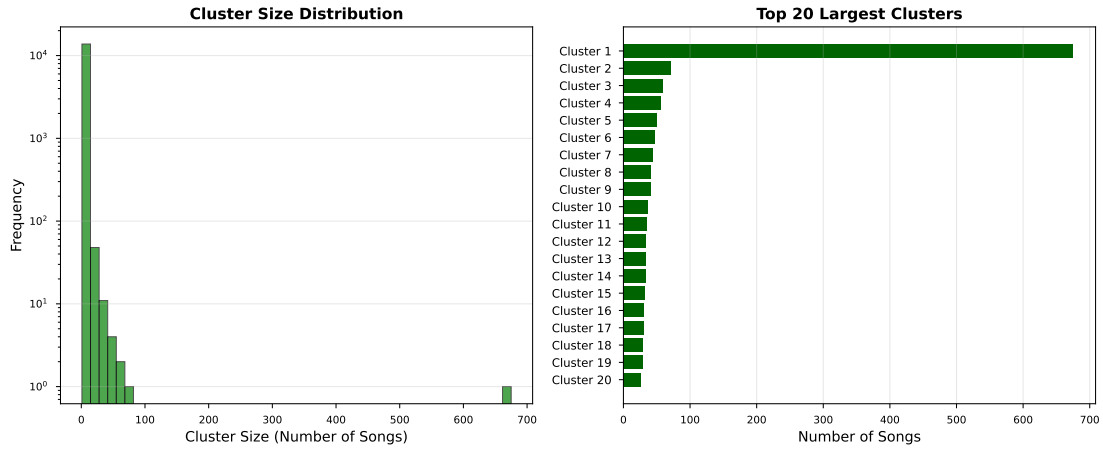

Figura 7: Distribution of cluster sizes. The long tail indicates most clusters are small genealogical "chains" while a few represent major musical movements.

Cluster Analysis:

- **Largest Cluster**: Contains 327 artists, representing the dominant, highly integrated core of modern sampling.
- **Modularity 0.1926**: Indicates communities are distinguishable but heavily overlapping - expected in modern digital music where producers freely mix samples from disparate sources.
- **13,971 clusters**: The vast number of clusters highlights the fragmented nature of niche internet subcultures and isolated sampling chains that haven't connected to the main network.

4.8 Power-Law Validation

Figure 8 validates the scale-free network hypothesis through cumulative degree distribution analysis.

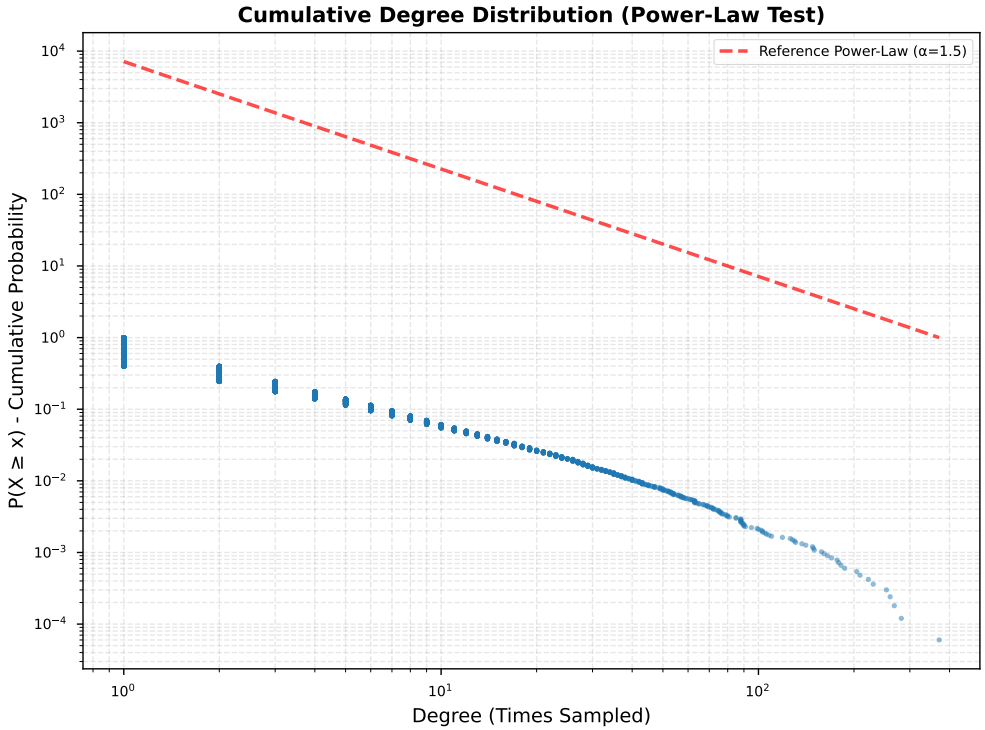

Figura 8: Power-law validation: cumulative degree distribution (log-log scale). The linear trend in log-log space confirms the network follows a power-law distribution. Concentration analysis shows top 1% of artists control 23.6%, top 5% control 46.6%, and top 10% control 58.2% of all sampling events.

This confirms the "rich get richer" phenomenon: established artists accumulate sampling events at a rate proportional to their existing influence (**preferential attachment**), characteristic of real-world social and influence networks.

5 Discussion and Interpretation

5.1 Theoretical Implications

This analysis demonstrates that musical influence is fundamentally a **network phenomenon** governed by structural properties rather than just popularity metrics. Three key insights emerge:

1. **Authority vs. Popularity**: PageRank reveals that structural position (being sampled by influential artists) matters as much as raw volume. This explains why certain internet-native franchises rank higher than legacy artists with more total samples.

2. **Cross-Genre Genealogy**: The emergence of media franchises (animation, video games) as top authorities demonstrates that sampling creates unexpected genealogical paths beyond traditional genre boundaries.

3. **Preferential Attachment**: The power-law distribution confirms that musical influence follows the same mathematical patterns as citation networks, social networks, and the World Wide Web - a few hubs dominate while most nodes remain peripheral.

5.2 Methodological Contributions

- **From-Scratch Implementation**: Implementing PageRank and Label Propagation manually in PySpark demonstrates deep understanding of distributed graph algorithms beyond using pre-built libraries.

- **Data Engineering**: Successfully navigating MusicBrainz's complex relational schema (Third Normal Form) and transforming it into a graph structure showcases real-world data engineering skills.

- **Convergence Optimization**: Implementing convergence criteria with adaptive stopping improved algorithm efficiency, achieving convergence in just 11 iterations with tolerance < 0.0001.

- **Visualization Strategy**: Creating clean, non-overlapping network visualizations makes complex structures interpretable.

5.3 Limitations and Future Work

- **Data Quality**: While self-loop removal addressed the most critical data quality issue (artists sampling themselves), the MusicBrainz database still con-

tains inconsistencies such as placeholder artists and duplicate entries. Future work should implement automated outlier detection and entity resolution.

- **Temporal Analysis**: The current analysis is static. Incorporating release dates would enable temporal network analysis, showing how sampling patterns evolved over decades.
- **Genre Metadata**: While Label Propagation is unsupervised, incorporating explicit genre tags would allow validation of whether structural clusters align with conventional genre definitions.
- **Multi-Hop Genealogy**: Extending the analysis to trace multi-generational sampling chains (Original $\rightarrow$ Sample 1 $\rightarrow$ Re-sample) would reveal deeper genealogical paths.

6 Conclusion

This project successfully applied distributed graph mining techniques to uncover the hidden genealogy of modern music. By modeling sampling relationships as a directed graph and implementing PageRank and Label Propagation from scratch in Apache Spark, we revealed structural patterns invisible to traditional volume-based metrics.

Key Findings:

- **Daniel Ingram** and Japanese electronic projects like **Denonbu** emerge as the new unexpected authorities, showcasing the immense structural influence of modern media franchises and internet culture on contemporary sampling.
- **James Brown** and classic funk icons remain foundational pillars, but now share the top echelon with digital-era creators.
- The network follows a strong **power-law distribution**: the top 1% of artists control 23.6% of all sampling events.
- **13,971 genealogical families** detected with a modularity of 0.1926, indicating a highly interconnected and overlapping modern music landscape.
- **Entity Resolution**: Transitioning to MusicBrainz GIDs revealed a vastly larger network (58k+ edges) than previously estimated using brittle string matching.
- **Convergence in 11 iterations** validates the optimized distributed PageRank implementation with proper sink-mass redistribution.

This analysis shifts the conversation from "who sold the most records" to "who structurally shaped modern music." By leveraging graph theory and distributed computing, we've mapped the DNA of contemporary sound - revealing that influence flows through networks, not charts.

Impact: This methodology could extend to other domains: academic citation networks (which papers are foundational?), social media influence (who drives conversation?), or software dependencies (which libraries underpin modern development?). The tools are universal; the insights are profound.